

MEMORIA ESCRITA DE INVERPEP

(Documentación técnica)

1. Resumen ejecutivo

InverPep es una aplicación web orientada a la consulta y exploración de péptidos antimicrobianos de invertebrados. El sistema permite visualizar registros, realizar búsquedas avanzadas, consultar detalles de cada péptido y acceder a información complementaria, como estadísticas descriptivas y material de ayuda.

Su arquitectura está compuesta por un **front-end estático servido con Nginx**, una **API REST implementada en Node.js con Express** y una **base de datos MySQL 5.7**. Adicionalmente, el sistema incorpora un mecanismo de respaldo mediante un **archivo FASTA local** para el endpoint de secuencias.

2. Objetivos

- Consultar de forma estructurada una base de datos de péptidos antimicrobianos.
- Permitir búsquedas por múltiples criterios biológicos y fisicoquímicos.
- Facilitar la revisión detallada de cada registro.
- Ofrecer una interfaz web sencilla, accesible y de bajo costo operativo.
- Mantener un mecanismo alternativo de consulta de secuencias mediante FASTA local.

3. Alcance

- Consulta general del conjunto de péptidos.
- Búsqueda avanzada por **código, especie, nombre, secuencia, tamaño, estructura y actividad**.
- Visualización detallada de registros individuales.
- Consulta de estadísticas descriptivas del dataset.
- Acceso a una sección de ayuda con glosario, explicación de cálculos y enlaces de apoyo.
- Respaldo de secuencias desde archivo FASTA cuando la consulta principal falla.

4. Arquitectura

- **Frontend (Nginx)**
 - HTML, CSS y JavaScript estático (**frontend/**).
 - Rutas amigables para navegación entre módulos.

- Compresión gzip habilitada.
- **API (Node.js/Express) (`api/server.js`)**
 - Endpoints REST para consulta de péptidos, detalle, búsqueda, salud del sistema y secuencias.
 - Conexión a MySQL mediante `mysql2`.
 - Fallback a FASTA en el endpoint de secuencias cuando falla la consulta principal.
- **Base de datos**
 - **Preferencia:** tabla `peptido` en MySQL 5.7.
 - **Alternativa:** archivo FASTA local (`api/sequences/BD_FINAL.fasta`) para el endpoint de secuencias.
- **Contenedores**
 - `docker-compose.yml` orquesta `mysql57`, `api` y `frontend` en la red

5. Estructura del repositorio

Repositorio: <https://github.com/HinaraSM12/Inverpep-page>

```
InverPep/
├── api/
│   ├── sequences/
│   │   └── BD_FINAL.fasta
│   ├── .env
│   ├── db.js
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── index.html
│   ├── help.html
│   ├── statistics.html
│   ├── listpeptidos.html
│   ├── searchpeptidos.html
│   ├── seepeptidos.html
│   ├── help.js
│   ├── listpeptidos.js
│   ├── searchpeptidos.js
│   ├── seepeptidos.js
│   └── styles.css
├── docker-compose.yml
├── nginx.conf
└── README.md
```

Descripción de cada archivo y carpeta

Raíz

- **docker-compose.yml**
Orquesta los servicios `mysql157`, `api` y `frontend`, así como sus dependencias, red y volúmenes.
Importante: requiere la red externa `inverpep_default` previamente creada.
- **nginx.conf**
Configura Nginx para servir el frontend, resolver rutas amigables y reenviar las solicitudes `/api/` al contenedor de la API.
Importante: también define compresión gzip y políticas básicas de caché.
- **README.md**
Documento general del proyecto con instrucciones de despliegue, configuración y uso.

api/ (servicio Node/Express)

- **sequences/BD_FINAL.fasta**
Archivo de respaldo para servir secuencias cuando la consulta principal a base de datos no está disponible.
- **.env**
Archivo de configuración local con variables de entorno del servicio API.
Importante: no debería versionarse con credenciales reales en producción.
- **db.js**
Gestiona la conexión y las consultas a MySQL mediante `mysql2`.
- **Dockerfile**
Define la construcción de la imagen de la API.
- **package.json**
Incluye dependencias y scripts del servicio.
- **server.js**
Archivo principal del servidor Express.
Implementa endpoints, filtros de búsqueda, consultas a la base de datos y lógica de respaldo FASTA.
Importante: concentra la lógica principal de la API.

frontend/ (cliente web servido por Nginx)

- **index.html**
Página principal de la plataforma.
- **help.html**
Centro de ayuda con contenido explicativo, glosario y enlaces.
- **statistics.html**
Vista de estadísticas descriptivas del dataset.
- **listpeptidos.html**
Listado general de péptidos con filtros rápidos.
- **searchpeptidos.html**
Formulario de búsqueda avanzada.
- **seepeptidos.html**
Vista de detalle de un péptido.

- **Archivos JavaScript asociados**
Implementan la lógica de cliente para búsqueda, listado, detalle y ayuda.
- **styles.css**
Define la apariencia visual general del sistema.

6. Flujo de datos

- El usuario abre una ruta del frontend y Nginx entrega el recurso HTML correspondiente.
- El JavaScript del cliente realiza solicitudes a `/api/...`
- Nginx reenvía la petición al contenedor `api`.
- Express procesa la solicitud y consulta MySQL.
- En el caso de `GET /api/sequences`, si la consulta principal falla, la API utiliza el archivo FASTA local como respaldo.
- El frontend renderiza la respuesta para el usuario.

7. API

GET /health

- Comprueba el estado general del sistema, incluyendo conectividad con la base de datos y existencia del archivo FASTA configurado.

GET /api/info

- Retorna información básica del servicio, como nombre, versión, base de datos, tabla y puerto.

GET /api/peptides

- Obtiene un conjunto de registros de la tabla `peptide`, ordenados por código.
Uso principal: poblar el listado general de péptidos.

GET /api/peptides/search

- Permite realizar búsqueda avanzada construyendo dinámicamente condiciones SQL a partir de los parámetros de consulta.
- **Filtros admitidos**
 - `codigo`
 - `especie`
 - `nombrE`
 - `secuencia`
 - `tamano`
 - `estr`
 - `act`

- **Importante:** usa consultas parametrizadas y limita la cantidad de resultados devueltos.

GET /api/peptides/:codigo

- Consulta un registro específico a partir de su clave primaria.
Uso principal: mostrar el detalle de un péptido.

GET /api/sequences

- Intenta recuperar secuencias desde MySQL. Si la consulta falla, procesa el archivo FASTA local, normaliza el contenido y responde una lista de secuencias válidas.

8. Configuración (variables de entorno)

API (api/.env o docker-compose.yml)

```
PORT=3000
DB_HOST=mysql57
DB_PORT=3306
DB_DATABASE=prospeccionydis
DB_USERNAME=prospeccion
DB_PASSWORD=biomole1520
DB_SSL=false
TABLE_NAME=peptido
FASTA_PATH=/app/sequences/BD_FINAL.fasta
FORCE_FASTA_ONLY=false
```

Descripción general

- **PORT** define el puerto interno del servicio API.
- **DB_HOST**, **DB_PORT**, **DB_DATABASE**, **DB_USERNAME** y **DB_PASSWORD** configuran la conexión a MySQL.
- **DB_SSL** habilita o deshabilita conexión segura.
- **TABLE_NAME** define la tabla principal consultada.
- **FASTA_PATH** especifica la ruta del archivo FASTA de respaldo.
- **FORCE_FASTA_ONLY** aparece en la configuración, aunque en la lógica actual no altera completamente el flujo.

Nginx

Configurado mediante `nginx.conf` para servir frontend, resolver rutas y redirigir `/api/` al contenedor `api`.

9. Despliegue

En la raíz del proyecto

```
docker network create inverpep_default  
docker compose up -d --build
```

Verificación sugerida

```
docker compose ps  
docker compose logs -f mysql57  
docker compose logs -f api  
docker compose logs -f frontend
```

10. Rendimiento y decisiones

- Frontend sin framework para reducir complejidad operativa.
- API compacta con lógica centralizada en `server.js`.
- Consultas parametrizadas con `mysql2`.
- Uso de Nginx para servir archivos estáticos y actuar como proxy inverso.
- Fallback FASTA para mantener disponibilidad básica del endpoint de secuencias.
- Las estadísticas se presentan como contenido estático y no se calculan dinámicamente en tiempo real.

11. Seguridad

- Uso de consultas parametrizadas para reducir riesgo de inyección SQL.
- Separación por contenedores para frontend, API y base de datos.
- Existencia de riesgos operativos por credenciales en archivos versionados.
- CORS configurado sin restricción específica de origen en el estado actual.
- En producción, se recomienda servir por HTTPS y externalizar secretos.

12. Respaldo y restauración

Elementos críticos de respaldo

- Volumen `mysql57_data`.
- Dump de base de datos.
- Archivo `api/sequences/BD_FINAL.fasta`.
- Configuración `api/.env`.
- Archivo `nginx.conf`.

Recomendaciones

- Exportar periódicamente la base de datos.
- Mantener copia del FASTA y de los archivos estáticos.
- Guardar la configuración fuera del contenedor.

- Si se requiere reprovisionar completamente MySQL desde scripts iniciales, puede ser necesario eliminar el volumen persistente existente.

13. Mantenimiento

- Revisar periódicamente los logs de los servicios.
- Verificar el endpoint `/health`.
- Actualizar imágenes base de Node y Nginx.
- Considerar migración desde MySQL 5.7 a una versión más reciente cuando el entorno lo permita.
- Pagar endpoints si el dataset crece.
- Revisar el uso real de la variable `FORCE_FASTA_ONLY` para evitar configuraciones confusas.

14. Limitaciones conocidas

- No hay pruebas automatizadas en el repositorio.
- La variable `FORCE_FASTA_ONLY` no se aplica completamente en la lógica actual.
- Existen credenciales expuestas en archivos versionados.
- No hay paginación en endpoints principales.
- Las estadísticas pueden quedar desactualizadas si cambia la base de datos.
- El proyecto depende de una red Docker externa preexistente.

15. Licencia y autores

Licencia

Este proyecto se distribuye bajo la **Licencia MIT**. Puedes usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del software, siempre que mantengas el aviso de copyright y esta nota de licencia en las copias o partes sustanciales del software. No ofrece garantías.

Autores y créditos

- **Autora principal:** Hinara Pastora Sánchez Mata.
Información de contacto: 3044822338, hisanchezm@unal.edu.co
- **Crédito institucional:** “Desarrollado por la línea de interacciones moleculares — Biología Funcional”.